

Lecture 1
Java Threads

Eng. Radu MIRON, PhD Senior Lecturer
radu.miron@aut.utcluj.ro

Java Threads

DOCUMENTATION:
https://docs.oracle.com/javase/tutorial/essential/concurrency/
https://jenkov.com/tutorials/java-concurrency/index.html#what-is-multithreading

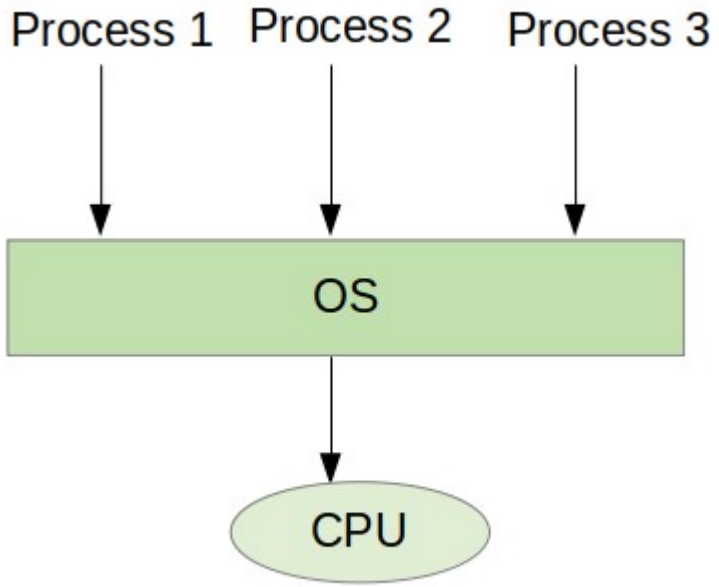
Threads allow the execution of several tasks (activities) at the same time. → *multithreading*

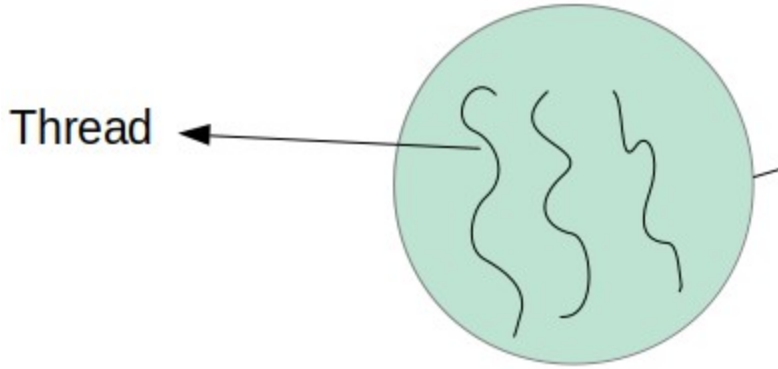
Threads != processes.

Processes are self contained execution environment (private memory space). → *multitasking*

Multiple threads (at least one) can exist in the same process sharing resources (memory and open files).

Multitasking vs. Multithreading

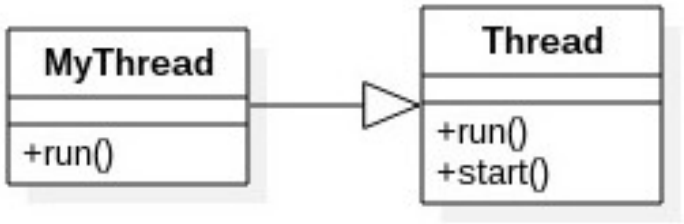




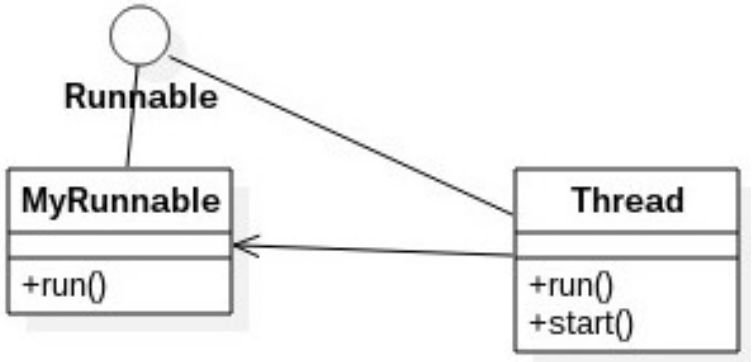
source: https://programmerbay.com/difference-between-multitasking-and-multithreading/

Implementation

Inheritance (class *Thread*)



Interface implementation *(Runnable)*





Thread Inheritance

```
public class Main {
   public static void main(String[] args) {
      MyThread t1 = new MyThread();
      MyThread t2 = new MyThread();
      t1.start();
      t2.start();
   }
}
class MyThread extends Thread {
   @Override
   public void run() {
      for (int i = 0; i < 10; i++) {
         System.out.println(
            Thread.currentThread().getName()
            + " msg " + i);
         try {
            Thread.sleep(100); // we will come back to this
         } catch (InterruptedException e) { // and this
         }
      }
   }
}
```



Don’t call run() to start the Threads!

```
public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.run(); // NO!
        t2.run(); // NO!
    }
}
class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(Thread.currentThread().getName() + "
      msg " + i);
            try {
                Thread.sleep(100); // we will come back to this
            } catch (InterruptedException e) { // and this
            }
        }
    }
}
```



Runnable Implementation

```
public class Main {
    public static void main(String[] args) {
        new Thread(new MyRunnable()).start();
        new Thread(new MyRunnable()).start();
    }
}

class MyRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(
                Thread.currentThread().getName()
                + " msg " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
            }
        }
    }
}
```



sleep() and interrupt()

```
class SlowExec extends Thread{
    @Override
    public void run() {
        try {
            Thread.sleep(1000 * 60 * 60 * 24); // 1 day
        } catch (InterruptedException e) {
            System.out.println("I've been interrupted");
        }
    }
}
class Intrerruptor extends Thread{
    private SlowExec slowExec; //...constructor association
    @Override
    public void run() {
        try {
            Thread.sleep(5000); // 5 sec
        } catch (InterruptedException e) {}
        slowExec.interrupt();
    }
}
```



Threads’ Priority

How to use?

t1.setPriority(n); t1.getPriority();
t1 – Thread object
n = 1..10

Default value: 5

What is its utility?
Through different priorities, developers can influence which threads receive CPU time preferentially, thereby optimizing performance and responsiveness for specific tasks. Threads get more time on CPU for greater values.



Threads' Name

How to use?

t1.setName("MyThread-1"); t1.getName();

t1 – Thread object

Default value:

main
Thread-0
Thread-1
etc.

What is its utility?

It increases the maintainability and debuggability of multi-threaded applications, making it possible for developers to trace execution, analyze logs and diagnose issues.



Mutual exclusion

Mutual exclusion refers to a concept in computer science and concurrent programming where **concurrent processes or threads are prevented from simultaneously accessing a shared resource** such as a variable, memory location, or data structure. The purpose of mutual exclusion is to ensure that only one process or thread can access the shared resource at any given time, thus preventing race conditions and maintaining data integrity.



Synchronized - block

```
public class MyThread extends Thread {
   private Integer lock;

   public MyThread(Integer lock) {
      this.lock = lock;
   }

   public void run() {
      System.out.println("Activity 1");
      synchronized (lock) {
         System.out.println("Critical Activity 2");
      }
      System.out.println("Activity 3");
   }
}

//-----------------somewhere in main() method
Integer lock = 1; // we have to use the SAME OBJECT
new MyThread(lock).start();
new MyThread(lock).start();
```



Synchronized - block

```
public class MyThread extends Thread {
   private static final Object LOCK = new Object();

   public void run() {
      System.out.println("Activity 1");
      synchronized (LOCK) {
         System.out.println("Critical Activity 2");
      }
      System.out.println("Activity 3");
   }
}

//----------------somewhere in main() method
new MyThread().start();
new MyThread().start();
```



Synchronized - block

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("Activity 1");
        synchronized (MyThread.class) {
            System.out.println("Critical Activity 2");
        }
        System.out.println("Activity 3");
    }
}

//----------------somewhere in main() method
new MyThread().start();
new MyThread().start();
```



Synchronized - method

```
public class Activity {
   public synchronized void doActivity() {
      System.out.println("Critical Activity 2");
   }
}
public class MyThread extends Thread {
   private Activity activity;
   public MyThread(Activity activity) {
      this.activity = activity;
   }
   public void run() {
      System.out.println("Activity 1");
      activity.doActivity();
      System.out.println("Activity 3");
   }
}
//-----------------somewhere in main() method
Activity activity = new Activity(); // we have to use the SAME OBJECT
new MyThread(activity).start();
new MyThread(activity).start();
```



It works for different methods of the same instanstance

```
public class FileAccess {
    private File f = new File("myFile.txt");
    private PrintWriter pw = new PrintWriter(new FileWriter(f, true));
    private BufferedReader br = new BufferedReader(new FileReader(f));

    public FileAccess() throws IOException {
        f.createNewFile();
    }

    public synchronized void writeLine(String line) {
        pw.println(line);
        pw.flush();
    }

    public synchronized String readLine() throws IOException {
        if (br.ready()) {
            return br.readLine();
        } else {
            return null;
        }
    }
}

// ... somewhere in main()
FileAccess fileAccess = new FileAccess(); // same instance for both threads
new WriterThread(fileAccess).start();
new ReaderThread(fileAccess).start();
```



Deadlocks

A **deadlock** in computer science occurs when **two or more processes are unable to proceed because each is waiting for the other to release a resource**, resulting in a circular waiting dependency. In Java, deadlocks can occur when multiple threads acquire locks on different resources and then attempt to acquire additional locks in a different order.

See examples in course repository.



```
radu@radu-pc:~$ jps
5216 SonarLintServerCli
42326 Jps
4921 Main
41884 Launcher
41886 DeadlockMain
4879
radu@radu-pc:~$ jstack 41886
2024-04-18 19:08:29
Full thread dump OpenJDK 64-Bit Server VM (21.0.2+13-Ubuntu-122.04.1 mixed mode
 sharing):
```

run command 'jps' to find PID

searched PID

run command jstack <PID>

. . .

```
Found one Java-level deadlock:
=============================
"Thread-0":
  waiting to lock monitor 0x00007f91ec002be0 (object 0x0000000717809f28, a java
.Object),
  which is held by "Thread-1"

"Thread-1":
  waiting to lock monitor 0x00007f91e0003900 (object 0x0000000717809f18, a java
.Object),
  which is held by "Thread-0"

Java stack information for the threads listed above:
===================================================
"Thread-0":
        at edu.tucn.ispse.lecture10.ex3deadlock.MyThread3.run(MyThread3.java:29
        - waiting to lock <0x0000000717809f28> (a java.lang.Object)
        - locked <0x0000000717809f18> (a java.lang.Object)
"Thread-1":
        at edu.tucn.ispse.lecture10.ex3deadlock.MyThread3.run(MyThread3.java:29
        - waiting to lock <0x0000000717809f18> (a java.lang.Object)
        - locked <0x0000000717809f28> (a java.lang.Object)

Found 1 deadlock.
```



Wait / notify

In Java, the **wait()** and **notify()** methods are part of the synchronization mechanism provided by the language for inter-thread communication and coordination. These methods are typically used in conjunction with the **synchronized** keyword to enable threads to wait for a condition to be met and to notify other threads when that condition changes.



Wait / notify

```
public class Main {
   static final Object monitor = new Object();
   public static void main(String[] args) throws IOException {
      new NotifierThread().start();
      new WaiterThread().start();
   }
}

public class WaiterThread extends Thread {
   public void run() {
      synchronized (Main.monitor) {
         try {
            Main.monitor.wait();
         } catch (InterruptedException e) {
         }
         System.out.println("I was notified");
      }
   }
}
```



Wait / notify

```
public class NotifierThread extends Thread {
  public void run() {
    try {
      System.out.println("Lets make the waiter wait");
      Thread.sleep(2000);

      synchronized (Main.monitor) { // uses the same instance
        Main.monitor.notify();
        System.out.println("The waiter will wait for the next sleep. " +
                           "I'm still holding the lock (still in 'synchronized')");
        Thread.sleep(2000);
      }
    } catch (InterruptedException e) {}
  }
}
```



Join

In Java, the **join()** method is used **to wait for a thread to complete its execution** before continuing with the execution of the current thread. It allows one thread to wait for another thread to finish before proceeding.



Join – code seq. 1

```
public class T1 extends Thread {
    public void run() {
        System.out.println("let T2 wait 5s");
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {}
        System.out.println("I'm done");
    }
}
```

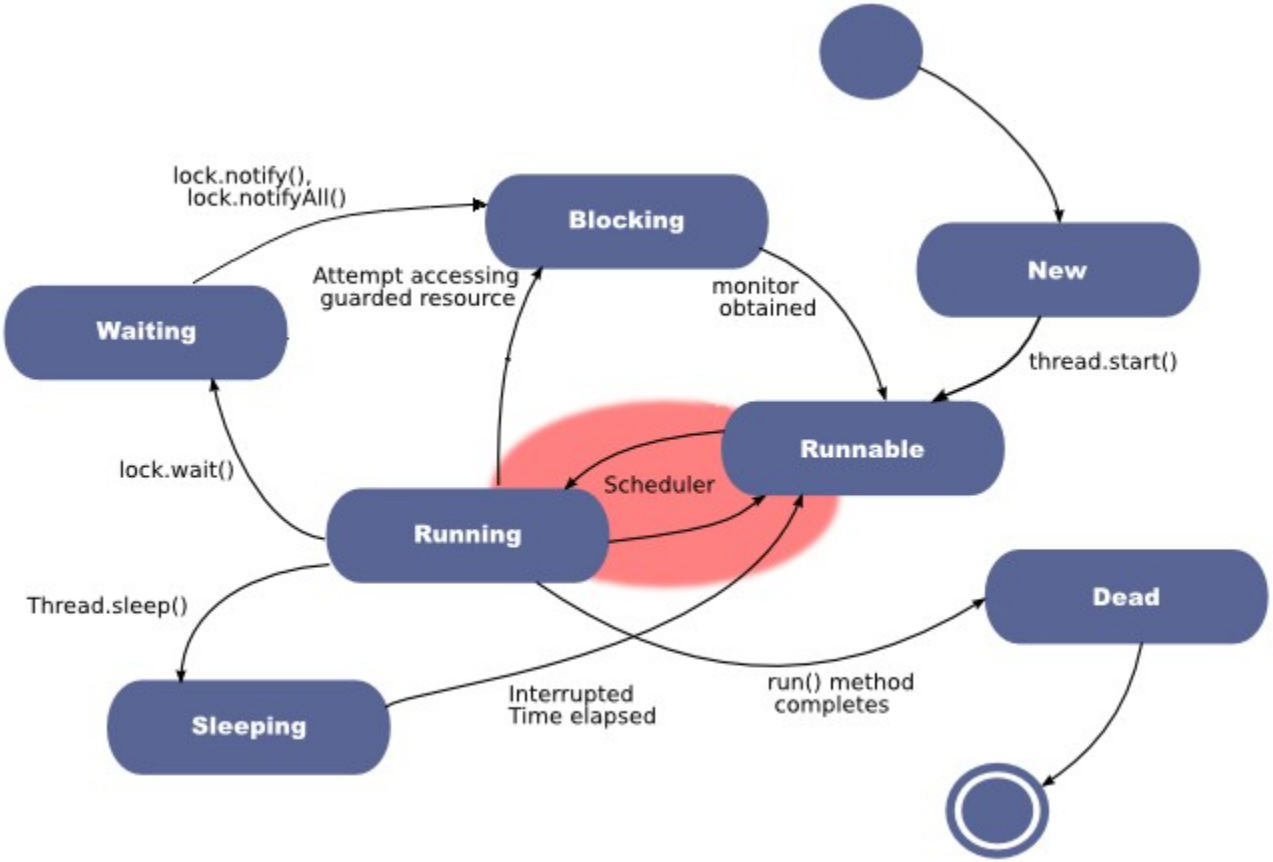


Join – code seq. 2

```
public class T2  extends Thread {
    private Thread t1;
    public T2(Thread t1) {
        this.t1 = t1;
    }
    public void run() {
        try {
            t1.join();
        } catch (InterruptedException e) {
        }
        System.out.println("I'm also done");
    }
}

//------------------------somewhere in main() method
Thread t1 = new T1();
t1.start();
new T2(t1).start();
```

The States of a Thread



source: http://booxs.biz/EN/java/Threads%20in%20Java.html



The end!